Data-Efficient Cross-Domain Few-Shot Website Fingerprinting With Unsupervised Domain Adaptation

Siyu Wang, Xiaoyan Hu, *Member, IEEE*, Guang Cheng, *Member, IEEE*, Ruidong Li, *Senior Member, IEEE*, and Hua Wu, *Member, IEEE*

***Abstract*—Website fingerprinting (WF) attacks identify Tor-encrypted websites but struggle with cross-domain scenarios due to traffic distribution shifts. The existing few-shot WF attacks address the cross-domain problem with excessive auxiliary data, significantly reducing deployment efficiency. This work proposes UDA-WF, a data-efficient few-shot WF with Unsupervised Domain Adaptation (UDA). UDA-WF first pre-trains the feature extractor with limited auxiliary data in the source website domain. Then, it extracts the invariant feature space by computing the intersection of the source and target feature spaces through the unsupervised domain adaptation with the softmatch mechanism. Finally, UDA-WF fine-tunes the feature extractor and a single-layer perceptron to extract the discriminative unique feature space of the target website domain. We evaluate UDA-WF on our WF dataset collected over multiple months. UDA-WF significantly overcomes the cross-domain problem while reducing auxiliary data requirements by 95% and pre-training bootstrap time by 99% compared to the State-Of-The-Art (SOTA) methods. UDA-WF achieves an accuracy of 97.37% under the 20-shot setting in the closed-world scenario and outperforms SOTA methods. To further demonstrate the model's adaptability to diverse real-world requirements, we validate it on the DF and Wang datasets, achieving accuracies exceeding 92% and 94%, respectively. Moreover, the results show that our UDA-WF is more resilient to concept drift and robust to WF defense.**

***Index Terms*—Website fingerprinting, cross-domain, few-shot learning, unsupervised domain adaptation, softmatch mechanism.**

Received 29 March 2025; revised 14 September 2025; accepted 7 May 2026. Date of publication 18 May 2026; date of current version 26 May 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2023YFB3106801, in part by the National Natural Science Foundation of China under Grant 62472087, in part by Jiangsu Province Natural Science Foundation Project under Grant BK20231413, and in part by the Fundamental Research Funds for the Central Universities under Grant 2242025K30025. The associate editor coordinating the review of this article and approving it for publication was Dr. Grigorios Loukides. *(Corresponding author: Xiaoyan Hu.)*

Siyu Wang, Guang Cheng, and Hua Wu are with the School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China (e-mail: wsy1323472153@seu.edu.cn; chengguang@seu.edu.cn; hwu@seu.edu.cn).

Xiaoyan Hu is with the School of Cyber Science and Engineering, Southeast University, Nanjing 211189, China, also with Purple Mountain Laboratories for Network and Communication Security, Nanjing 211111, China, also with Jiangsu Provincial Engineering Research Center of Security for Ubiquitous Network, Nanjing 211189, China, and also with the Engineering Research Center of Blockchain Application, Supervision and Management, Ministry of Education, Nanjing 211189, China (e-mail: xyhu@njnet.edu.cn).

Ruidong Li is with the Institute of Science and Engineering, Kanazawa University, Kakuma, Kanazawa 920-1192, Japan (e-mail: liruidong@ieee.org).

Digital Object Identifier 10.1109/TIFS.2026.3694650

I. Introduction

ANONYMOUS communication systems protect user privacy by preventing tracking and identification. Tor [1], a widely used system with 200 million daily users, secures browser data through multi-layer encryption and randomly routed circuits. However, passive attacks like website fingerprinting (WF) can still de-anonymize users by analyzing unique traffic patterns. Prior research [2], [3], [4], [5], [6], [7], [8] has demonstrated the effectiveness of WF attacks, achieving over 95% accuracy. However, the requirement for considerable training data limits their applicability to quickly adapt to newly monitored conditions, i.e., the cross-domain problem. The cross-domain problem [9] occurs when the distributions of the source and target domains are different, such as using different versions of the Tor Browser Bundle (TBB) and having different monitored requirements.

Few-shot WF attacks have been proposed to address the cross-domain problem with limited target domain data. Few-shot learning [10] is a machine learning paradigm that enables models to quickly adapt to new tasks or recognize new classes with only a small amount of labeled data. Most existing few-shot WF attacks [11], [12], [13], [14], [15], [16] share a similar design architecture to solve the cross-domain problem. They first pre-train their general feature extractors with substantial auxiliary website data (source domain) and then fine-tune them with limited labeled target website data to identify the target website trace (target domain). The WF feature space consists of an invariant feature space (such as Tor cell padding patterns, similar websites, encrypted channel construction strategy) agnostic to the WF task and unique feature spaces (such as the TBB version, network condition, unique websites) specific to individual datasets. The pre-training phase extracts the invariant feature space of the WF task from the source website dataset, and the fine-tuning phase extracts the unique feature space specific to the target website trace. Due to the particular data input representation of the WF task, the invariant and unique feature spaces of the source website domain are linearly inseparable. Therefore, the existing few-shot WF attacks require substantial auxiliary website data (source domain data) for pre-training, leading to time-consuming and complex pre-training processes and significantly reducing the deployment efficiency of few-shot WF attacks.

1556-6021 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:23:13 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SUMMARY OF REPRESENTATIVE WF ATTACKS

Phase	Methods	Attack Characteristics			Training and auxiliary dataset description[c]	Training and auxiliary dataset requirements
		Cross-domain	Data-efficient[a]	Resilient adaptation[b]		
Traditional WF attacks	AWF [5]	✗	✗	✗	AWF100_2500	250000
	DF [6]	✗	✗	✗	DF95_1000	95000
	Tik-Tok [7]	✗	✗	✗	DF95_1000	95000
	RF [8]	✗	✗	✗	DF95_1000	95000
Few-shot WF attacks	TF [11]	✓	✗	✗	AWF775_25	19375
	WFBDC [14]	✓	✗	✗	AWF775_25	19375
	NetCLR [15]	✓	✗	✗	AWF100_500	50000
	CF [22]	✓	✗	✗	CF100_500	50000
	DCWF [23]	✓	✗	✗	AWF775_25	19375
	UDA-WF	✓	✓	✓	**AWF100_25**	**2500**

[a] Data-efficient represents that the attack needs low data requirements (both the training data and the auxiliary data).
[b] Resilient adaptation represents that the attack can adapt to changes in monitored requirements with low training/pre-training bootstrap time.
[c] Training and auxiliary dataset description represents dataset usage. Traditional WF attacks use these datasets to train their model. Few-shot WF attacks use auxiliary datasets to pre-train their feature extractors. The dataset description includes the dataset name, the number of websites, and the number of training/auxiliary instances per website. For example, "AWF775 25" means the few-shot attack uses 775 websites from the AWF dataset and 25 instances per website to pre-train its model.

To address the above issues, we propose a data-efficient few-shot WF attack based on Unsupervised Domain Adaptation (UDA) with the softmatch [17] mechanism called UDA-WF[1] (Unsupervised Domain Adaptation based Website Fingerprinting) to overcome the cross-domain problem with limited auxiliary data requirements. First, UDA-WF pre-trains the feature extractor to fit the source feature space and decision boundaries, where Global Average Pooling (GAP) reduces feature dimensionality and classifier parameters, thereby lowering auxiliary data requirements. Second, UDA-WF leverages UDA with the softmatch mechanism to generate and weight pseudo-labels from clustering unlabeled target data, guiding the feature extractor toward the intersection of source and target feature spaces. Finally, UDA-WF fine-tunes the feature extractor and a single-layer perceptron to extract the discriminative unique feature space of the target website dataset. The contributions of our work can be summarized as follows:

- We propose a data-efficient few-shot website fingerprinting model, UDA-WF, based on UDA with the softmatch mechanism. It enhances the feature extraction ability of the feature extractor with GAP, significantly reducing classifier parameters and the training dataset requirement. Our UDA with softmatch forces target features to reside in the source decision regions through the pseudo-label technique and down-weights unreliable pseudo-labels by the softmatch mechanism, ensuring that the learned representation corresponds to a purer domain-invariant feature space. The UDA process uses only the unlabeled form of the fine-tuning data, ensuring fairness in performance comparison with other few-shot WF models while avoiding additional auxiliary data.
- We collect a comprehensive dataset from Majestic,[2] the WF dataset, to evaluate the performance of attack models when encountering a significant cross-domain between the source and target website datasets. The monitored websites in our WF dataset differ from those in the source dataset by 86%, and the TBB version is very different from the pre-training dataset.
- We prototype and compare our UDA-WF with the representative few-shot attacks across different website datasets. UDA-WF achieves the best accuracy of 97.37% under the 20-shot setting in the WF dataset, outperforming the SOTA few-shot WF attacks, WFBDC, DCWF, and NetCLR, by 18.97%, 30.24%, and 2.79% in the closed-world scenario. Even on the DF and Wang datasets, our model achieves the best accuracy scores of 92.48% and 94.62% in the closed-world scenario. Our UDA-WF also outperforms SOTA methods in the open-world scenario. Moreover, the results show that our UDA-WF is more resilient to concept drift and robust to WF defense. The experimental results also illustrate that our UDA-WF can overcome the cross-domain problem with limited auxiliary data requirements. Compared to the SOTA methods, the auxiliary data requirements and pre-training bootstrap time are reduced by 95% and 99%, respectively.

[1]https://github.com/Lwsy123/UDA-WF
[2]https://blog.majestic.com/development/majestic-million-csv-daily/

II. RELATED WORK

This section reviews the existing traditional and few-shot WF attacks and WF defenses. Table I summarizes representative WF attacks.

A. Traditional WF Attacks

Traditional WF attacks can be classified as manual feature engineering and automated feature engineering. Manual feature engineering WF attacks resort to Machine Learning (ML) [2], [3], [4], [18], such as KNN, Random Forest, and SVM, showing their effectiveness in WF attacks. However, the manual feature engineering-based WF attacks are significantly sensitive to traffic dynamics, limiting their recognition performance in real-world WF attacks. Automated feature engineering WF attacks [5], [6], [7], [8], [19], [20], [21] are based on Deep Learning (DL), remarkably improving over

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:23:13 UTC from IEEE Xplore. Restrictions apply.

ML-based WF attacks. For example, Rimmer et al. [5] proposed Automated Website Fingerprinting (AWF). AWF proves that the DL-based models (SDAE, CNN, LSTM) can automatically learn traffic features. It achieves over 96% accuracy in the closed-world scenarios. Sirinam et al. [6] proposed Deep Fingerprinting (DF). DF extracts the traffic features through CNN and achieves an accuracy of over 98%, far exceeding the performance of manual feature engineering. Rahman et al. [7] proposed the Tik-Tok model. Tik-Tok combines the time and direction features to enhance the performance of the model against WF defense. It demonstrates that the time features are effective against WF defense. Shen et al. [8] proposed the RF model. It uses a novel input representation, TAM, against the WF defense methods, achieving the most robust results. Zhou et al. [19] proposed WF-Transformer, which enhances the feature extraction ability through the attention-based feature extractor and achieves an accuracy of over 99%. However, the requirement for substantial training data prevents DL-based WF attacks from quickly adapting to changes in monitored conditions.

B. Few-Shot WF Attacks

Few-shot WF attacks have been proposed to adapt to changes in monitored conditions quickly. We divide them into three groups based on their domain-invariant feature extraction techniques.

1) Fitting-Based Methods: Fitting-based methods derive domain-invariant features solely from source-domain samples by minimizing the distance between intra-class features while enlarging the separation among inter-class features. Sirinam et al. [11] proposed the TF model. It utilizes a triplet network to pre-train the general WF feature extractor, achieving satisfactory results with only 25 traces per website. Zou et al. [14] proposed the WFBDC model. It uses Brownian Distance Covariance to pre-train the feature extractor and reduce the pre-training time. Moreover, it achieves significant improvements in recognition performance across multiple datasets. Bahramali et al. [15] proposed the NetCLR model. It resorts to unsupervised learning to pre-train a general WF feature extractor and achieves the SOTA performance. Xie et al. [22] proposed the Contrastive Fingerprinting (CF). CF applies contrastive loss to build an efficient feature extractor and extracts the global features of encrypted traffic. However, fitting-based methods derive domain-invariant features solely from source data, often introducing source-unique features absent in the target domain and requiring extensive data due to the inseparability of invariant and unique features, significantly increasing computational cost and pre-training time.

2) Alignment-Based Methods: Alignment-based methods derive domain-invariant features from target and source-domain samples by minimizing the distance between target and source feature spaces. Luo et al. [23] proposed the DCWF model. It introduces a new feature extractor, the circle network, and overcomes the cross-domain problem through two-stage distribution calibration, significantly improving the recognition performance. Lyu et al. [24] proposed the TFAN model. It follows a meta-learning paradigm, formulating WF attacks as a class-wise feature alignment problem. TFAN demonstrates strong performance across various scenarios, particularly under concept drift conditions. However, alignment-based methods require simultaneous access to source and target data, increasing computational overhead. Besides, since it minimizes overall distributional distance rather than just invariant features, domain-specific features may also be aligned, contaminating the extracted invariant feature space and reducing performance.

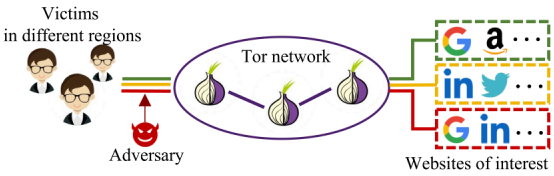


Fig. 1. Threat model. Victims in different regions have different websites of interest. The adversary trains region-specific WF attacks through few-shot learning.

3) Federated-Based Methods: Federated-based methods do not train feature extractors but instead integrate existing feature extractors. Tan et al. [25] proposed a federated learning-based framework, SMC. It integrates the feature spaces of multiple client-side attackers to enhance the representation capability of local models, achieving promising recognition performance. However, since it builds on features extracted by existing client-side attack models, its performance is inherently limited by the capabilities of those models.

C. WF Defenses

WF defenses [26], [27], [28], [29], [30], [31] obfuscate original website traffic patterns to counter WF attacks. Gong and Wang [27] proposed Front, inserting highly random dummy packets at the beginning of each trace. It significantly reduces the recognition performance of WF attacks. De la Cadena et al. [28] proposed TrafficSliver. It divides user traffic into multiple entries and limits the data within a single entry node. It can reduce the WF attack accuracy from more than 98% to less than 16%. Ling et al. [30] proposed WFGuard. It generates padding strategies with the fuzzing testing, significantly decreasing the accuracy of DL-based WF classifiers (e.g., DF and Var-CNN).

III. Threat Model

Figure 1 illustrates our threat model. We assume the victims are in different regions, and the adversary launches our few-shot WF attack to monitor the victims.

A. Victims

We assume the victims originate from different regions, such as countries or cities. Victims in different regions have different websites of interest. They access websites using privacy-enhancing techniques like Tor to hide their online activities. Their Tor browsers or on-path Tor relay nodes may employ WF defense mechanisms to minimize the de-anonymization risk.

B. Adversary

The adversary controls the victims' local network devices but cannot obtain content due to Tor encryption, collecting

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:23:13 UTC from IEEE Xplore. Restrictions apply.

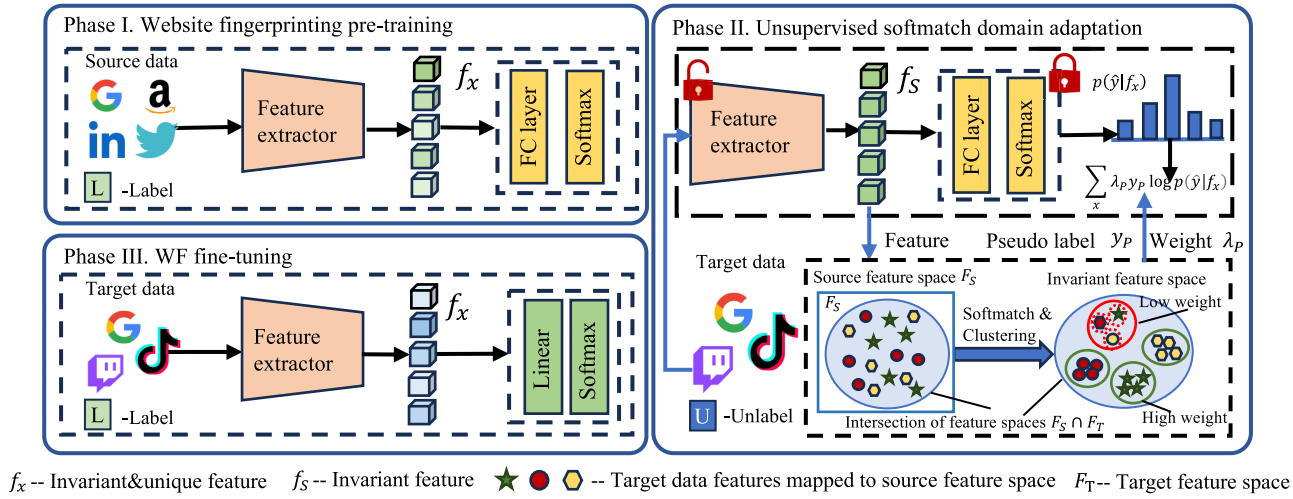


Fig. 2. UDA-WF framework. 1) Extract the source website feature space by pre-training the feature extractor. 2) Adapt the pre-trained feature extractor by extracting the invariant feature space from the source feature space through unsupervised softmatch domain adaptation. 3) Employ the adapted feature extractor to extract the unique feature space of the target website dataset.

only Tor cell traces. To de-anonymize client activities, the adversary infers visited websites via WF. However, regional differences in websites of interest and TBB versions limit the availability of training data. The adversary uses few-shot learning to quickly fine-tune and deploy region-specific WF attack models with minimal data to address this.

We consider both closed-world and open-world scenarios. In the closed-world setting, victims browse only monitored websites to evaluate the performance of WF attacks under controlled scenarios. In the open-world scenario, victims also visit non-monitored websites to evaluate the performance of WF attacks when facing unexpected websites.

IV. System Design

This section presents the design details of our UDA-WF. We start with an overview of UDA-WF and then elaborate on its modules.

A. Overview

As discussed in Section II, the prior few-shot WF attacks require substantial auxiliary data to pre-train due to extracting the invariant feature space from the source feature space. It is challenging to extract the invariant feature space with limited auxiliary data since the particular data input representation of the WF task makes the invariant and unique feature spaces linearly inseparable. Moreover, the linear inseparability makes it difficult to completely exclude the unique feature space from the extracted invariant feature space, where the impure invariant feature space reduces the effectiveness in addressing the cross-domain problem.

Our UDA-WF overcomes the cross-domain problem with limited auxiliary data requirements. First, it pre-trains the feature extractor with limited auxiliary data to obtain the source feature space. Second, it freezes the classifier weights to serve as fixed decision boundaries in the source domain. It leverages unsupervised domain adaptation with the softmatch mechanism to extract the invariant feature space. UDA generates pseudo-labels through clustering unlabeled target website data and weights the pseudo-labels based on softmatch mechanism. These weighted pseudo-labels then guide the update of the feature extractor, driving it toward the intersection of the source and target feature spaces (i.e., the domain-invariant feature space). Third, it fine-tunes the feature extractor and a single-layer perceptron to extract the discriminative unique feature space of the target website dataset. Figure 2 illustrates the overall architecture of UDA-WF. It consists of the following three modules.

1) Website Fingerprinting Pre-Training: The website fingerprinting pre-training leverages limited auxiliary data to extract the source feature space, which contains the invariant feature space. Since invariant and unique features are linearly inseparable, directly isolating the invariant part from the source space requires substantial data. Our objective at this stage is to train the feature extractor with limited auxiliary data to obtain the source feature space. In addition, we incorporate GAP into the extractor to improve efficiency. GAP not only enhances the generalization of the model but also reduces the size of the classifier parameters, thereby lowering the data requirement to fit the complete feature space of the source.

2) Unsupervised Softmatch Domain Adaptation: Since the extracted source feature space contains invariant and unique features, UDA generates pseudo-labels through clustering to guide the update of the feature extractor on unlabeled target website data, thus driving it toward the intersection of the source and target feature spaces (i.e., the domain-invariant feature space). However, pseudo-labels near cluster boundaries are often unreliable and may hinder the extracting of informative invariant features. To mitigate this issue, we incorporate the softmatch mechanism to assign confidence weights to pseudo-labels and down-weight low-quality ones, thereby encouraging the extractor to learn a purer domain-invariant representation.

3) WF Fine-Tuning: With the extracted invariant feature space, the WF fine-tuning module fine-tunes the feature extractor with the limited target dataset to obtain the discriminative

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:23:13 UTC from IEEE Xplore. Restrictions apply.

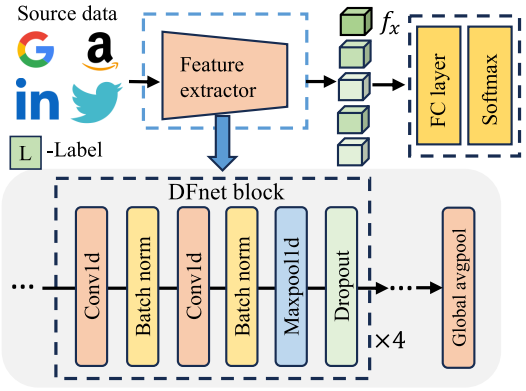


Fig. 3. Structure of website fingerprinting pre-training.

unique feature space of the target website dataset and overcome the cross-domain problem between the source and target feature space.

B. Website Fingerprinting Pre-Training

The website fingerprinting pre-training module obtains the complete invariant feature space while minimizing the source domain's unique feature space. Since the invariant feature space contains features common to data samples, it can be fitted with a small amount of data. Thus, we pre-train the website feature extractor with limited auxiliary data, significantly reducing auxiliary data requirements.

Figure 3 shows our WF feature extractor model, the DFnet. The DFnet consists of 4 DFnet blocks with channel sizes of [32, 64, 128, 256], and each DFnet block contains two 1D CNN layers, two batch normalization layers, a maxpooling layer, and a dropout layer. In particular, we add a GAP layer after 4 DFnet blocks to integrate the extracted features without increasing the number of parameters. Feature integration can reduce spatial dimensions and improve the generalization of the model. It also reduces the spatial dimensions of the classification layer parameters. By reducing the spatial dimensions, UDA-WF can lower computational costs and use less auxiliary data to fit the complete source feature space and decision boundaries.

We pre-train the feature extractor on the source website dataset (such as the AWF dataset) by selecting M traces for each website (we discuss the selection of M in the following experiments) to minimize the following cross-entropy loss, as shown in Equation 1.

$$\mathcal{L}_{cls}(f; \mathcal{X}_s, \mathcal{Y}_s) = -\mathbb{E}_{(x_m, y_m)\in \mathcal{X}_s \times \mathcal{Y}_s} \sum_{m=1}^{M} \sum_{k=1}^{K} \mathbb{I}_k(y_m)\, log\delta_k(f(x_m)) \quad (1)$$

Here, $f(\cdot) : \mathcal{X} \rightarrow R^K$ represents the DFnet where $\mathcal{X}$ is the input data, and K is number of source monitored website URLs. $\mathcal{X}_s$ and $\mathcal{Y}_s$ denote the input website traces and labels of the source website domain. $\mathbb{E}(\cdot)$ is the average operation. (x_m, y_m) is one of the website trace and label pairs derived from the source website dataset. $\mathbb{I}_k(\cdot)$ is the indicator function taking the value of 1 if $y_m = k$ or 0 otherwise. $\delta_k(\cdot)$ is the softmax function. Let $\alpha_m = f(x_m)$. $\delta_k(\alpha_m) = \frac{exp(\alpha_{m,k})}{\sum_{i=1}^{K} exp(\alpha_{m,i})}$ where $\alpha_{m,i}$ denotes the i-th component of α_m.

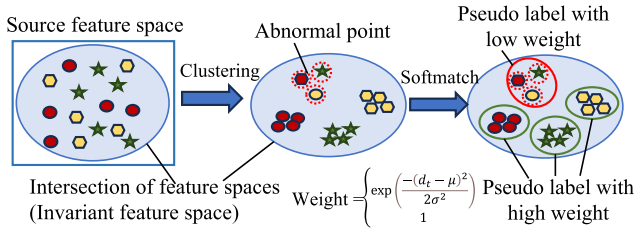


Fig. 4. Pseudo-label generation based on softmatch. The softmatch mechanism weighs the pseudo-labels generated by the cluster boundary points (abnormal points) with low weights to reduce the negative impact of them for UDA.

C. Unsupervised Softmatch Domain Adaptation

Since the extracted source feature space consists of the invariant and unique feature spaces, the unsupervised softmatch domain adaptation module is responsible for extracting the pure invariant feature space from the source feature space through UDA with the softmatch mechanism.

UDA [32], [33], [34] relies on pseudo-label generation technology to guide the feature extractor towards the intersection of source and target feature space during the domain adaptation. Here, we generate pseudo-labels based on clustering. Ideally, cluster boundaries should be distinguishable. However, cluster boundaries have some less distinguishable points generated by WF defense and network fluctuations (abnormal points) in reality (as shown in Figure 4). Abnormal points substantially reduce the reliability of pseudo labels, thereby contaminating the extracted invariant feature space with unique domain features. Therefore, we resort to the softmatch [17] mechanism to generate pseudo-label weights to address this problem, where the pseudo-labels of cluster boundaries have far lower weights than the pseudo-labels closed to cluster centers. As a result, the unsupervised softmatch domain adaptation module is divided into two submodules: invariant feature space extractor with UDA and pseudo-label generation based on the softmatch mechanism.

1) Invariant Feature Space Extractor With UDA: We extract the intersection of the source and target feature spaces to obtain the invariant feature space. In this submodule, we freeze the classifier $h(\cdot)$ of the DFnet $f(\cdot)$ to train the CNN-based feature extractor $g(\cdot)$ through UDA, where the frozen weights of the classifier serve as fixed decision boundaries in the source domain. We extract the intersection of the source and target website feature space using N unlabeled target data $\mathcal{X}_t$ for UDA to minimize the cross-entropy loss function, as shown in Equation 2.

$$\min_{h(g(\cdot))} \mathcal{L}_{cls}(f; \mathcal{X}_t; \mathcal{Y}_p) = -\mathbb{E}_{(x_n, y_n)\in \mathcal{X}_t \times \mathcal{Y}_p} \sum_{n=1}^{N} \sum_{k=1}^{K} \mathbb{I}_k(y_n)\, log\delta_k(h(g(x_n))) \quad (2)$$

where $\mathcal{Y}_p$ represents the pseudo-label, and (x_n, y_n) is one of the target website trace and pseudo-label pairs.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:23:13 UTC from IEEE Xplore. Restrictions apply.

We freeze the parameters of the classifier $h(\cdot)$. Then, the minimization of the objective function depends only on the feature extractor $g(\cdot)$, as shown in Equation 3.

$$\begin{aligned} &\min_{g(\cdot)} \mathcal{L}_{cls}(f; \mathcal{X}_t; \mathcal{Y}_p) \\ &= -\mathbb{E}_{(x_n, y_n) \in \mathcal{X}_t \times \mathcal{Y}_p} \sum_{n=1}^{N} \sum_{k=1}^{K} \mathbb{I}_k(y_n) \, log \delta_k(\, h(g(x_n))) \end{aligned} \tag{3}$$

The frozen classifier $h(\cdot)$ defines the decision regions of the source domain. It preserves the discriminative structure learned from the source data. Pseudo labels describe the potential distribution of the target feature space within source decision regions. Only target features consistent with the source discriminative patterns can be effectively represented. Target-unique features outside these regions cannot be reinforced and are gradually suppressed. As the classification loss L_{cls} decreases during training, the feature extractor is guided to retain increasingly pure invariant features. Moreover, when training the feature extractor $g(\cdot)$ with unlabeled target samples, the optimization gradient can be expressed as Equation 4.

$$\Delta u_n \;\propto\; -J_h(u_n)^{\top}(p_n - y_n), \tag{4}$$

where $u_n = g(x_n)$ is the target feature, $p_n = \delta(h(u_n))$ is the predicted probability, y_n is the pseudo label, and $J_h(u_n)$ denotes the Jacobian of the frozen classifier $h(\cdot)$ with respect to u_n. Namely, each u_n is pulled towards the discriminative direction of its pseudo-labeled class and pushed away from other class directions, ensuring that the extractor emphasizes domain-invariant features while suppressing unique source domain noise. Source-unique features absent from X_t are not reinforced during training. Consequently, the optimization is naturally guided towards the intersection of source and target feature space, resulting in a purer feature representation rather than noisy feature alignment and feature fitting.

2) Pseudo-Label Generation Based on Softmatch: The pseudo-label generation maps the label space of the target website domain to the label space of the source website domain, facilitating invariant feature extraction, as shown in Figure 4.

Clustering algorithms are one of the most common methods to generate pseudo-labels [35], [36], [37] in UDA. However, there is a trade-off between quality and quantity when generating pseudo-labels.

- **Quality of pseudo-labels.** If preserving all generated pseudo-labels, the pseudo-labels of cluster boundaries introduce more noise generated by WF defenses and network fluctuations into the invariant feature space, negatively affecting the extraction of invariant features.
- **Quantity of pseudo-labels.** If setting a higher confidence threshold to improve pseudo-label quality, the quantity of pseudo-labels reduces, which may reduce the extracted invariant feature space area and thus exclude some invariant features.

Our softmatch-based pseudo-label generation algorithm makes a trade-off between quality and quantity. The softmatch mechanism employs a Gaussian function to weight the pseudo-label for each sample, assigning higher weights to pseudo-labels at cluster centers. Additionally, an Exponential Moving Average (EMA) continuously updates the weights to balance the quantity and quality of pseudo-labels. Below, we detail the softmatch pseudo-label generation algorithm, as shown in Algorithm 1.

Algorithm 1 Softmatch Pseudo-Label Generation

Input: dataset $\mathcal{D}(x_n, y_n)_{n=1}^{N}$, step rate m_{step}, generation indicator M_{gen}, label type K, pre-trained feature extractor $g(\cdot)$, pre-trained classifier $h(\cdot)$,
Output: pseudo-label $\mathcal{Y}_p$, pseudo-label weight λ_p

1 Initialize B, L as empty arrays, $j = 0$
2 **for** n = *1, 2, ..., N* **do**
3 select (x_n, y_n) from $\mathcal{D}$
4 $feature = g(x_n)$
5 add $feature$ in $B[j]$
6 $y_{pred} = softmax(h(feature))$
7 add y_{pred} in $L[j]$
8 $j = j + 1$
9 **end**
10 Initialize U as $K \times K$ identity matrix
`// U is the one-hot encoding of label`
11 $C_0 = \frac{L^T B}{\sum_i^N l_i}$ where $l_i \in L$
12 $\mu = 0 \quad \sigma^2 = 1$
13 **for** i *in range(1,* $M_{gen} - 1$*)* **do**
14 $\mathcal{Y}_p = argmin(1 - \frac{BC_{i-1}^T}{\|B\|_2 \|C_{i-1}\|_2})$
15 $d_t = min(1 - \frac{BC_{i-1}^T}{\|B\|_2 \|C_{i-1}\|_2})$
16 $\mu_a = \frac{1}{N} \sum_{z=1}^{N} d_t[z] \,, \sigma_a^2 = \frac{1}{N} \sum_{z=1}^{N} (d_t[z] - \mu_a)^2$
17 $\mu = m_{step}\mu + (1 - m_{step})\mu_a$
18 $\sigma = m_{step}\sigma + (1 - m_{step})\sigma_a$
19 $L = U[\mathcal{Y}_p]$
20 $C_i = \frac{L^T B}{\sum_{i=1}^{N} l_i}$ where $l_i \in L$
21 **end**
22 λ_p is calculated by Equation 6
23 Return $\mathcal{Y}_p$, λ_p

We calculate the cosine distance between features and cluster centers to measure the confidence of pseudo-labels. The cosine distance function D_f is defined as Equation 5.

$$D_f(B, C_{i-1}) = 1 - \frac{BC_{i-1}^T}{\|B\|_2 \; \|C_{i-1}\|_2} \tag{5}$$

Here, $D_f(B, C_{i-1})$ denotes the cosine distance, i.e., 1 - cosine similarity. This definition is consistent with the distance calculation in Algorithm 1 (lines 14-15). B represents the website trace feature map, and i is the $i-th$ iteration of cluster. C_{i-1} represents cluster centers of the features at the $(i-1)-th$ iteration. T is the matrix transpose operation.

We use a truncated Gaussian function λ_p to assign weights to the pseudo-labels, as shown in Equation 6.

$$\lambda_p(d) = \begin{cases} exp\left(-\dfrac{(d-\mu)^2}{2\sigma^2}\right), & \text{if } d > \mu \\ 1, & \text{otherwise} \end{cases} \tag{6}$$

where d represents the cosine distance between the data point and the corresponding cluster center. μ and σ represent the

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:23:13 UTC from IEEE Xplore. Restrictions apply.

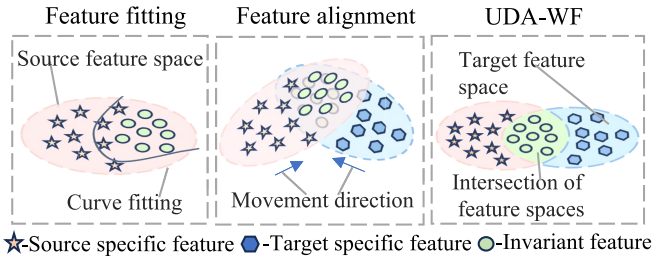


Fig. 5. Differences among feature fitting, feature alignment, and UDA-WF in extracting invariant feature space.

empirical mean and variance, respectively. The empirical mean μ indicates the average cosine distance from each sample point to the nearest cluster center, with a range of [0, 2], derived from the range of D_f. However, the empirical mean μ and variance σ of the Gaussian function are initially unknown. Therefore, we use the Exponential Moving Average (EMA) algorithm to calculate them. Specifically, we calculate the mean μ_t and variance d_t for each iteration (line 16) and use them to update μ and σ (lines 17-18). After several iterations, we obtain the pseudo-labels and their corresponding weights for the target domain data (line 22).

Moreover, we employ entropy minimization to improve the model performance when extracting the invariant feature space. In addition, we use Kullback-Leibler divergence to ensure that the distribution of the generated pseudo-labels is uniform. In practice, we minimize two loss functions $\mathcal{L}_{ent}$ and $\mathcal{L}_{div}$, as defined in Equation 7 and Equation 8.

$$\mathcal{L}_{ent}(f;\mathcal{X}_t) = -\mathbb{E}_{x_n\in\mathcal{X}_t}\sum_{n=1}^{N}\sum_{k=1}^{K}\delta_k(f(x_n))log\delta_k(f(x_n)) \tag{7}$$

$$\mathcal{L}_{div}(f;\mathcal{X}_t) = D_{KL}(p,u(K)) = \frac{1}{K}\sum_{k=1}^{K}p_k log\frac{p_k}{u(K)} \tag{8}$$

where $u(K)$ represents a uniform distribution, and $p_k = -\mathbb{E}_{x_n\in\mathcal{X}_t}\sum_{n=1}^{N}\delta_k(f(x_n))$ denotes the average probability of each source website label predicted by the model.

To minimize the negative impact of these unreliable labels during the UDA phase, we introduce the weights generated by the softmatch mechanism to the original cross-entropy loss function. Equation 9 defines the weighted cross-entropy loss.

$$\mathcal{L}_{cls}(f;\mathcal{X}_t,\mathcal{Y}_p,\lambda_p) = -\mathbb{E}_{(x_n,y_n,\lambda_n)\in\mathcal{X}_t\times\mathcal{Y}_p\times\lambda_p}\sum_{n=1}^{N}\sum_{k=1}^{K}\lambda_n\mathbb{I}_k(y_n)\,log\delta_k(\,f(x_n)) \tag{9}$$

Here, λ_p represents the weight of the pseudo-label $\mathcal{Y}_p$. (x_n, y_n, λ_n) is one triplet of the target trace, pseudo-label, and weight pairs. Equation 10 defines the total loss function.

$$\mathcal{L} = \beta\mathcal{L}_{cls}(f;\mathcal{X}_t,\mathcal{Y}_p,\lambda_P) + \mathcal{L}_{ent}(f;\mathcal{X}_t) + \mathcal{L}_{div}(f;\mathcal{X}_t) \tag{10}$$

where $\beta > 0$ is a balancing hyper-parameter.

Figure 5 illustrates the difference among feature fitting, feature alignment, and UDA-WF in extracting the invariant feature space. The feature fitting methods obtain the invariant feature space by fitting with extensive auxiliary data. The feature alignment methods obtain the invariant feature space by reducing the distance between the source and target feature spaces. Therefore, when optimizing the feature extractor, they must use source and target data, increasing computing costs. To reduce the auxiliary data requirements and pre-training bootstrap time, our UDA-WF obtains the intersection of the source and target website feature space (i.e., the pure invariant feature space). The UDA method is SHOT-inspired [38], and we use the softmatch mechanism to enhance it. The UDA process uses only the unlabeled form of the fine-tuning data, ensuring fairness in performance comparison with other few-shot WF models while avoiding additional auxiliary data. Moreover, collecting unlabeled target traces is considerably easier than obtaining labeled ones.

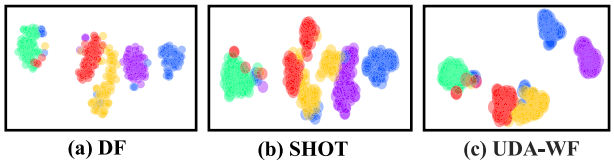


Fig. 6. t-SNE visualizations of DF, SHOT, and UDA-WF for 5-way 20-shot WF tasks.

Figure 6 illustrates the t-SNE visualizations of DF, SHOT, and UDA-WF for 5-way 20-shot WF tasks. Compared to DF, SHOT's domain adaptation strategy aggregates features from different datasets into an unknown class as much as possible. SHOT [38] does not map the target domain to the source domain. It is difficult to extract the intersection. In contrast, the domain adaptation strategy of our UDA-WF model clusters the data and maps different classes to specific categories in the source domain so that the intersection of the source domain and the target domain can be obtained.

D. WF Fine-Tuning

Although we have obtained the pure invariant feature space from the intersection of the source and target feature spaces, the lack of a target unique feature space, such as version-specific TBB-exclusive behavior and unique websites, hinders the feature extractor from overcoming the cross-domain problem. Therefore, the WF fine-tuning module extracts the unique feature space of the target dataset to overcome the cross-domain problem.

Figure 7 shows the structure of the WF fine-tuning. The WF fine-tuning module employs the adapted feature extractor to extract the unique feature space of the target website feature space. A simple classifier, such as a single-layer perceptron, is sufficient for classification if the extracted unique feature is discriminative. Therefore, we fine-tune the feature extractor and a single-layer perceptron to extract the discriminative unique feature space. We use N-shot website instances to fine-tune the model. The selected N-shot fine-tuning instances are the same as the unlabeled data used for unsupervised domain adaptation. Finally, the fine-tuned feature extractor and perceptron classifier are employed to predict unknown traffic.

V. Dataset Setup

This section presents the datasets used in our study. We categorize our datasets into pre-training, cross-domain, large

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:23:13 UTC from IEEE Xplore. Restrictions apply.

TABLE II
DATASET DETAILS

Dataset	TBB Version	Origin	Collection Time	Monitor	Non-monitor	Purpose		
						Pre-tain	UDA	Fine-tune
AWF Dataset [5]	6.X	Alexa	2017	100	0	✓	✗	✗
WF Datset	13.X	MAJESTIC	2024	90	9000	✗	✓	✓
DF Dataset [6]	7.X	Alexa	2017	95	9000	✗	✓	✓
Wang Dataset [2]	3.X	Alexa	2013	100	9000	✗	✓	✓

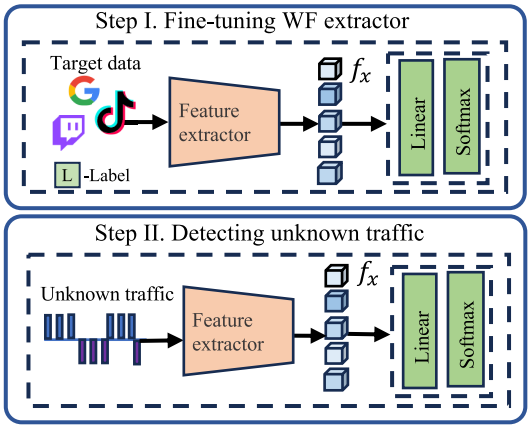


Fig. 7. Structure of WF fine-tuning.

open-world, concept drift, and defense datasets for different experimental requirements. Table II describes these datasets. Since the AWF, WF, Wang, and DF have huge differences, we use them to evaluate the cross-domain performance of WF attack models. The differences among the four datasets can be expressed as follows.

- Various TBB versions introduce noticeable differences in website traffic patterns. For example, TBB 3.x (2013) is based on an older Firefox ESR core and lacks modern padding strategies, producing relatively simple traffic traces. In contrast, TBB 6.x and 7.x (2017) adopt updated encryption libraries and introduce more stable cell padding, which alters packet size and timing distributions. TBB 13.x (2024) recently supports modern web standards and HTTP/2 with more aggressive traffic padding, leading to richer, longer, and more dynamic traffic traces. These version-specific discrepancies contribute to significant cross-domain shifts across datasets.
- The four datasets differ in their monitored and non-monitored websites, which poses a key challenge for cross-domain WF attacks. For example, the monitored websites in the WF dataset are selected from Majestic rather than Alexa, and the different ranking criteria lead to a distinct set of monitored websites. In addition, the continuous evolution of Internet technologies and web ecosystems causes website content to change over time. Consequently, although AWF, Wang, and DF are all collected based on Alexa rankings, their monitored websites are not identical, and the WF dataset is even more distinct as it was collected in 2024 using Majestic rankings. These temporal and source discrepancies result in noticeable variations in traffic patterns. Specifically, the Wang dataset, collected in 2013, reflects an early web landscape in which large-scale video streaming and cloud services were not yet widespread, leading to fewer multimedia-intensive websites. In contrast, the AWF and DF datasets, collected in 2017, capture a later stage of the ecosystem dominated by social media, e-commerce, and online video platforms, producing more complex traffic traces. Most recently, the WF dataset, collected in 2024, represents the modern Internet environment, characterized by richer multimedia content, highly interactive web applications, and diverse traffic behaviors.

Due to the shutdown of the Alexa website, we collect the WF dataset based on the ranking from the MAJESTIC website. The MAJESTIC project, which started in 2012 and is still being updated, ranks websites based on the number of referring subnets and includes one million websites. We use TBB 13.0.1 to access websites and capture their traffic. For web crawling, we use torbrowserselenium[3] (an optimized Selenium package for operating with the Tor browser) and stem[4] (a Tor control package), while capturing the traffic via the scapy[5] package. For the selection of monitored websites, we first choose the top 100 sites from the MAJESTIC rankings, then exclude sites with different top-level domains but identical second-level domains (e.g., google.com and google.de) as well as sites with access restrictions (sites that could not be accessed even after multiple link changes). Ultimately, we select 90 websites. When collecting data, we configure Selenium to close a website immediately after it is fully rendered in the browser and then proceed to the next one. We do not frequently change Tor circuits during this process since each circuit introduces latency, path length, and bandwidth characteristics. If changing the circuit too often, the resulting traffic traces would contain substantial circuit-induced noise, and the model might exploit these circuit patterns rather than learning website-unique features. To avoid this bias, our circuit update strategy keeps the same circuit within one full crawl of the website list and does not update it until the list has been completed or when the program is restarted. Such an update strikes a balance between consistency and diversity. It minimizes circuit-induced noise within each batch of data, preventing the model from exploiting circuit patterns to distinguish different websites, while introducing sufficient circuit variation across the entire dataset to ensure generalization. Since some websites restrict access via Tor, we take snapshots

[3]https://github.com/webfp/tor-browser-selenium
[4]https://github.com/torproject/stem
[5]https://github.com/secdev/scapy

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:23:13 UTC from IEEE Xplore. Restrictions apply.

of the web pages and train a CNN-based web snapshot filter to ensure that the data we collect is clean. Specifically, if a snapshot contains the content of the target website, we retain the traffic from that visit. Otherwise, we discard the traffic.

A. Dataset for Pre-Training

We select the AWF dataset [5] as the pre-training dataset, and it contains 100 websites collected from Alexa with TBB 6.5 in 2017. We choose M instances per website to form the pre-training dataset ($M = \{25, 50, 75, 100\}$).

B. Dataset for Cross-Domain

As shown in Table II, the WF and Wang datasets differ significantly from the AWF dataset in TBB versions and collection time. In contrast, the DF dataset is more similar to AWF, where the differences in the collection time and TBB version are minor.

The WF dataset includes 90 monitored websites from MAJESTIC and 9000 non-monitored websites, collected using TBB 13.0 in 2024. The Wang dataset contains 100 monitored websites and 9000 non-monitored websites, selected from Alexa in 2013 using TBB 3.5. The DF dataset consists of 95 monitored websites and 9000 non-monitored websites, sourced from Alexa in 2017 with TBB 7.0.6.

C. Dataset for Large Open-World

We use AWF-OW50k [5], AWF-OW100k, AWF-OW200k, and AWF-OW400k to evaluate the performance of our UDA-WF when facing large non-monitored websites. These datasets contain 50k, 100k, 200k, and 400k non-monitored websites from the AWF dataset, respectively.

D. Dataset for Concept Drift

We make WFdrift84 and WFdrift97, which contain traces of the same websites as the WF dataset, captured on the 84th and 97th days after the first capture.

E. Dataset for Defense

We evaluate the performance of our model when facing the Front [27] and TrafficSliver [28] defense methods. We get two datasets for defense generated by the WF dataset, the WF-front and WF-trafficsliver datasets.

VI. Performance Evaluation

This section evaluates the effectiveness of UDA-WF in different scenarios and also compares it with the SOTA few-shot WF attacks.

A. Experimental Setup

We conduct our experiments using Python 3.8 and PyTorch 1.14, running on an NVIDIA 3090Ti GPU.

1) Methodology: We conduct the subsequent experiments to evaluate the effectiveness of our proposed UDA-WF.

- **Evaluation of UDA-WF.** In this experiment, we conduct a comprehensive evaluation of our UDA-WF model in three parts. First, we evaluate the auxiliary data requirements for our UDA-WF. Second, we conduct an ablation study to evaluate the effects of the UDA and softmatch mechanism. Third, based on the above experiment results, we compare the auxiliary data requirements and pre-training bootstrap time with SOTA few-shot WF attacks.
- **Cross-domain in the closed-world and open-world scenarios.** The closed-world and open-world scenarios are two common experiments to evaluate the performance of WF attacks. Since few-shot WF attacks focus on the cross-domain problem, we evaluate the performance of few-shot WF attacks when facing the cross-domain problem in closed-world and open-world scenarios.
- **Large open-world scenario.** The large open-world scenario [11], [23] is a common scenario considered by few-shot WF attacks. Since there are various non-monitored websites in the real-world scenario, we need to evaluate the performance of few-shot WF attacks when facing a large number of non-monitored websites.
- **Concept drift.** Concept drift [5], [6] is a fundamental challenge that both traditional WF attacks and few-shot WF attacks must address. Concept drift occurs when the website content is changed (such as text, images, and advertisements). Due to changes in website content, the website traffic pattern changes dramatically, significantly degrading the performance of WF attacks. Since concept drift originates from changes in data distribution, it can also be classified as a cross-domain problem.
- **WF defense.** Since our threat model considers that the Tor browser and on-path Tor relay nodes can employ WF defense, we evaluate the robustness of our UDA-WF against WF defense methods. WF defense [26], [27], [28], [29], [31] disrupts the website traffic patterns through traffic shaping or obfuscation methods to counter the WF attacks.

2) Metrics: We evaluate the performance of few-shot WF in different scenarios from the following aspects.

- **Accuracy.** The proportion of correct predictions made by a model on a dataset. $Accuarcy = \frac{TP+TN}{TP+TN+FP+FN}$.
- **Precision.** The proportion of true positive instances among all instances predicted as positive by the model. $Precision = \frac{TP}{TP+FP}$.
- **Recall.** The proportion of true positive instances predicted by the model among all actual positive instances. $Recall = \frac{TP}{TP+FN}$.
- **F1-score.** The harmonic mean of precision and recall. $F1 = 2 \times \frac{Precision*Recall}{Precision+Recall}$.
- **AUC.** The area under the ROC curve is used as a performance measure in the open-world scenario. The ROC curve is generated by plotting the True Positive Rate (TPR) against the False Positive Rate (FPR). $TPR = \frac{TP}{TP+FN}$,$FPR = \frac{FP}{FP+TN}$.

TP represents the true positive. FP represents the false positive. TN represents the true negative. FN represents the false negative. Each experiment is repeated for 5 runs with different seeds to get the average results. We report the results in terms of mean $\pm$ standard deviation.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:23:13 UTC from IEEE Xplore. Restrictions apply.

TABLE III
HYPERPARAMETER SETTINGS OF UDA-WF

Parameters	Search Space	Choice
Input Dimension	[500,..7000]	5000
Optimizer	[Adam, SGD, AdamW]	Adam
Learning Rate	[0.001,...,0.01]	0.001
Mini-batch Size	[16,...,256]	128
Filter, Pool, Stride Sizes	[2,...,16]	[8.8,4]
Activation Function	[Tanh,ReLU,ELU]	ELU,ReLU
DFnet Block1 [Conv1,Conv2]	[8,...,64]	[32,32]
DFnet Block2 [Conv3,Conv4]	[32,...,128]	[64,64]
DFnet Block3 [Conv5,Conv6]	[64,...,256]	[128,128]
DFnet Block4 [Conv7,Conv8]	[128,...,512]	[256,256]
Pooling layer	[Average, Max]	Max
Dropout rate	[0.1,...,0.8]	0.1
generation indicator M_{get}	{1,5,10,20}	20
step rate m_{step}	{0.1-1}	0.5
DA epoch E_{DA}	{ 10, 30, 50, 70}	70
balancing hyperparamter β	{0.1-1}	1

3) Model Parameters: In the closed-world scenario, we use $N \in \{5, 10, 15, 20\}$ instances per website to fine-tune the benchmark models and 70 instances per website as the target test dataset. In the open-world scenario, we use $N \in \{5, 10, 15, 20\}$ instances per monitored website and $L \in \{500, 1000, 1500, 2000\}$ non-monitored websites to fine-tune the benchmark models. The monitored and non-monitored instances are combined with corresponding combinations (i.e., $\{(N_i, L_i) | N_i \in N, L_i \in L\}$). Additionally, we use 70 instances per monitored website and 7000 non-monitored websites as the target test dataset. To simplify the notation, we use K-shot to describe the number of fine-tuning instances in closed-world and open-world scenarios. Table III describes the hyperparameter setting of UDA-WF.

4) Baselines: We select the most representative and recent few-shot WF attacks to compare with our UDA-WF.

- **Deep fingerprinting (DF)** [6]. Since our few-shot attack baselines' feature extractors originate from the DF model, we use DF as a baseline. The DF model uses a CNN-based feature extractor for WF.
- **Triplet fingerprinting (TF)** [11]. The TF model is one of the earliest few-shot website fingerprinting models. It uses a triplet network to facilitate few-shot learning. The main structure of TF consists of the DF model's feature extractor with a K-Nearest Neighbors (KNN) classifier.
- **WFBDC** [14]. WFBDC is one of the earliest few-shot website fingerprinting models. It extracts the domain-invariant feature space through Brownian Distance Covariance (BDC) to minimize the distance between intra-class features and enlarge the separation among inter-class features. The WFBDC model builds on the DF model and uses the BDC pooling to enhance it.
- **NetCLR** [15]. NetCLR is the current SOTA method for few-shot website fingerprinting. The NetCLR model builds on the original DF model and applies unsupervised learning for pre-training.
- **DCWF** [23]. DCWF is one of the latest few-shot website fingerprinting methods to overcome the cross-domain problem. It optimizes two-stage distribution calibration to overcome the cross-domain problem.

We retrain the DF, TF, DCWF, and NetCLR on the above datasets to ensure fair comparisons.

B. Evaluation of UDA-WF

In this section, we first evaluate the auxiliary data requirements and conduct the ablation study. Then, we compare UDA-WF with the baselines in time efficiency.

1) Auxiliary data requirements in website fingerprinting pre-training. In this experiment, we evaluate the UDA-WF's auxiliary data requirements for pre-training. We select the AWF dataset with M instances per website as the pre-training dataset, where $M \in \{25, 50, 75, 100\}$. The dataset contains 100 websites. To ensure generalizable conclusions, we investigate the impact of different pre-training data requirements on UDA-WF recognition performance across three benchmark datasets (WF, DF, and Wang datasets). Accuracy is used as the evaluation metric.

As shown in Figure 8, the accuracy curves for the three datasets remain almost flat across different fine-tuning instance settings. The average fluctuation of recognition performance is less than 0.06%, 0.06%, and 0.08% in the WF, DF, and Wang datasets. The results indicate that UDA-WF exhibits minimal sensitivity to variations in pre-training data requirements. Therefore, we use 25 instances per website to pre-train the feature extractor in the subsequent experiments to mitigate the dependency on auxiliary data requirements and reduce training overhead.

Remark 1: The experiment demonstrates that our UDA-WF requires minimal auxiliary data, significantly enhancing its deployment efficiency.

2) Ablation study. We investigate the effect of the unsupervised domain adaptation (UDA) and the softmatch mechanism. We evaluate them across three benchmark datasets (WF, Wang, and DF datasets). Accuracy is our evaluation metric.

As shown in Table IV, the UDA and the softmatch mechanism noticeably affect the recognition performance. The result shows an average accuracy reduction of 1.50% without a softmatch mechanism in the WF dataset, 0.70% in the Wang dataset, and 1.75% in the DF dataset. The average reduction is 1.50% without both UDA and softmatch mechanisms in the WF dataset, 0.99% in the Wang dataset, and 1.30% in the DF dataset. Moreover, we notice no significant difference in the recognition performance of UDA-WF between using only UDA and not applying UDA. The reason is primarily that, in UDA, pseudo-labels at the cluster edges and those at the cluster centers are assigned equal weights. As a result, a considerable amount of noise (such as the source TBB feature) is introduced into the extracted invariant feature space, which substantially compromises the effectiveness of UDA. The results indicate that the softmatch mechanism can enhance the performance of UDA, thereby improving the model's recognition accuracy.

Remark 2: This experiment validates the effectiveness of the UDA with the softmatch mechanism. We argue that the improvement brought by UDA with the softmatch mechanism is primarily due to the weighting of different pseudo-labels by softmatch, reducing the negative impact of pseudo-labels at the cluster edges on UDA. As a result, the extracted invariant

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:23:13 UTC from IEEE Xplore. Restrictions apply.

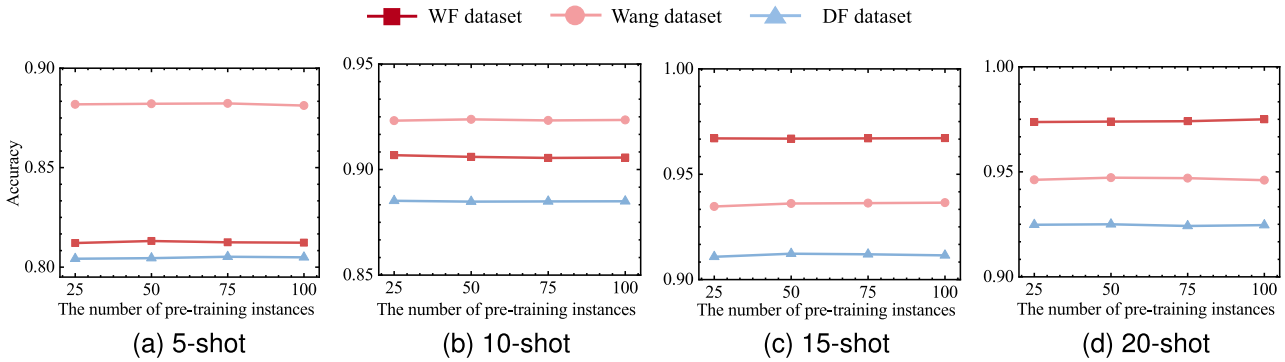


Fig. 8. Performance of UDA-WF over different auxiliary data requirements.

TABLE IV
ABLATION EXPERIMENTS OF OUR UDA-WF WITH DIFFERENT TARGET DOMAINS. ACCURACY IS OUR EVALUATION METRIC, AND ALL NUMBERS ARE %

Dataset	Ablation		K-shot			
	UDA	Softmatch[a]	5	10	15	20
WF dataset	✓	✓	**81.21 ± 0.49**	**90.68 ± 0.29**	**96.70 ± 0.14**	**97.37 ± 0.07**
	✓	✗	80.60 ± 0.35	87.16 ± 0.27	95.75 ± 0.31	96.97 ± 0.08
	✗	✗	80.75 ± 0.47	87.63 ± 0.55	95.70 ± 0.18	96.35 ± 0.11
Wang dataset	✓	✓	**88.18 ± 0.27**	**92.32 ± 0.09**	**93.47 ± 0.10**	**94.62 ± 0.06**
	✓	✗	86.92 ± 0.34	91.77 ± 0.18	93.10 ± 0.40	94.22 ± 0.11
	✗	✗	87.27 ± 0.24	91.35 ± 0.32	92.55 ± 0.07	93.77 ± 0.31
DF dataset	✓	✓	**80.42 ± 0.34**	**88.52 ± 0.14**	**91.08 ± 0.13**	**92.48 ± 0.10**
	✓	✗	78.84 ± 0.47	86.63 ± 0.21	89.58 ± 0.12	91.30 ± 0.10
	✗	✗	78.42 ± 0.34	87.82 ± 0.18	89.93 ± 0.29	91.85 ± 0.08

[a] Since the softmatch is used to optimize the UDA, we can not evaluate the effect of the softmatch mechanism independently.

TABLE V
EFFICIENCY COMPARISON WITH SOTA FEW-SHOT WF ATTACKS IN EXTRACTING THE INVARIANT FEATURE SPACE. ALL MODELS ARE TRAINED USING THE OPTIMAL PARAMETER SETTINGS SPECIFIED IN THE ORIGINAL REFERENCES

Model	Auxiliary dataset	Auxiliary requirements	Pre-training bootstrap time[a]
TF	AWF775_25	19375	762s
WFBDC	AWF775_25	19375	148s
NetCLR	AWF100_500	50000	113163s
DCWF	AWF775_25	19375	180s
UDA-WF	AWF100_25	**2500**	$\leq$ **94s**[b]

[a] Since the overhead of few-shot learning mainly lies in pre-training with a fixed dataset, we record just the pre-training bootstrap time.
[b] It is the sum of pre-training bootstrap time for website fingerprinting pre-training and unsupervised domain adaptation.

feature space contains less noise, enhancing the recognition performance of UDA-WF.

3) Time efficiency comparison. We set the pre-training parameters of UDA-WF following the above experiments. To ensure the reliability of results, the baseline models are pre-trained in strict accordance with the auxiliary data requirements and parameter configurations proposed in TF [11], WFBDC [14], NetCLR [15], and DCWF [23], as shown in Table V.

We further evaluate the time efficiencies of the existing few-shot WF attacks for extracting the invariant feature space. Since the UDA-WF model extracts the invariant feature space through two stages, website fingerprinting pre-training and UDA, the pre-training bootstrap time shown in Table V represents the total time for the two stages. However, as the unlabeled data requirements in the UDA stage vary, we evaluate and record the maximum value of our pre-training bootstrap time. Table V shows that with just 2500 instances for pre-training, UDA-WF reduces the data requirements by 87.09%, 87.09%, 95%, and 87.09% compared to TF, WFBDC, NetCLR, and DCWF. UDA-WF has the lowest pre-training time cost, with less than 94s, reducing the time cost by at least 87.66%, 36.48%, 99.91%, and 47.77% compared to TF, WFBDC, NetCLR, and DCWF.

Remark 3: This experiment illustrates that our UDA-WF can significantly reduce the pre-training bootstrap time and the auxiliary data requirements. The extraction of the intersection of source and target feature spaces instead of direct data fitting primarily contributes to the high data and time efficiency. High data and time efficiency characteristics make our model more suitable for the low-resource adversary.

C. Cross-Domain in Closed-World Scenario

We evaluate the performance of the UDA-WF when facing the cross-domain problem in the closed-world scenario. In this scenario, we compare our model with existing SOTA models, including DF, TF, WFBDC, DCWF, and NetCLR.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:23:13 UTC from IEEE Xplore. Restrictions apply.

TABLE VI

PERFORMANCE COMPARISON WITH SOTA FEW-SHOT WF ATTACKS IN THE CLOSED-WORLD SCENARIO WITH DIFFERENT TARGET DOMAINS. ACCURACY IS OUR EVALUATION METRIC, AND ALL NUMBERS ARE %

Dataset	Method	K-shot			
		5	10	15	20
WF dataset	DF	25.74 ± 1.95	31.23 ± 0.75	66.70 ± 0.43	65.57 ± 0.73
	TF	31.75 ± 0.27	37.82 ± 0.28	40.35 ± 0.60	44.30 ± 0.14
	WFBDC	53.90 ± 0.20	70.09 ± 0.15	77.22 ± 0.14	81.84 ± 0.08
	DCWF	41.25 ± 0.33	57.93 ± 0.71	68.60 ± 0.38	74.76 ± 1.12
	NetCLR	75.71 ± 0.30	88.89 ± 0.23	93.12 ± 0.29	94.72 ± 0.11
	UDA-WF	**81.21 ± 0.49**	**90.68 ± 0.29**	**96.70 ± 0.14**	**97.37 ± 0.07**
Wang dataset	DF	41.53 ± 1.26	64.95 ± 0.59	76.35 ± 0.65	79.30 ± 0.42
	TF	69.74 ± 0.35	73.36 ± 0.49	75.64 ± 0.22	76.24 ± 0.34
	WFBDC	85.64 ± 0.72	89.09 ± 0.28	91.48 ± 0.13	93.09 ± 0.17
	DCWF	85.92 ± 0.10	90.12 ± 0.18	91.80 ± 0.23	93.12 ± 0.42
	NetCLR	85.91 ± 0.23	90.72 ± 0.19	92.71 ± 0.17	93.84 ± 0.12
	UDA-WF	**88.18 ± 0.27**	**92.32 ± 0.09**	**93.47 ± 0.10**	**94.62 ± 0.06**
DF dataset	DF	38.12 ± 0.28	54.35 ± 0.26	61.32 ± 0.43	73.13 ± 0.07
	TF	55.64 ± 0.21	64.70 ± 0.13	65.20 ± 0.46	67.57 ± 0.32
	WFBDC	74.48 ± 0.79	83.12 ± 0.40	87.11 ± 0.48	89.71 ± 0.18
	DCWF	77.24 ± 0.11	85.05 ± 0.06	87.65 ± 0.26	90.30 ± 0.18
	NetCLR	79.81 ± 0.49	87.85 ± 0.13	89.80 ± 0.12	91.41 ± 0.10
	UDA-WF	**80.42 ± 0.34**	**88.52 ± 0.14**	**91.08 ± 0.13**	**92.48 ± 0.10**

We experiment on WF, Wang, and DF datasets, respectively. Accuracy is the evaluation metric.

WF dataset evaluation results. Table VI shows that UDA-WF outperforms the SOTA attack methods on the WF dataset. The accuracy of UDA-WF reaches 97.37% under the 20-shot setting, where the SOTA few-shot models, WFBDC, DCWF, and NetCLR, are 81.84%, 74.76%, and 94.72%, respectively. Even under the 5-shot setting, UDA-WF improves the accuracy over the baseline models, DF, TF, WFBDC, DCWF, and NetCLR, by 215.50%, 155.77%, 50.66%, 96.87%, and 7.26%.

Wang dataset evaluation results. On the Wang dataset, UDA-WF also outperforms the SOTA attack methods. As shown in Table VI, with 20 instances per website for fine-tuning, UDA-WF achieves an accuracy of 94.62%, where the SOTA few-shot methods, WFBDC, DCWF, and NetCLR, achieve accuracies of 93.09%, 93.12%, and 93.84%. Even under the 5-shot setting, UDA-WF achieves an accuracy of 88.18%, improving the accuracy over the baseline models, DF, TF, WFBDC, DCWF, and NetCLR, by 112.32%, 26.44%, 2.96%, 2.63%, and 2.64%.

DF dataset evaluation results. We also evaluate our model on the DF dataset. Table VI shows that UDA-WF achieves higher accuracy on the DF dataset. UDA-WF achieves an accuracy of 92.48% under the 20-shot setting, outperforming all baseline models, DF, TF, WFBDC, DCWF, and NetCLR, by 26.45%, 36.86%, 3.08%, 2.41%, and 1.17%.

The results show that our UDA-WF model can overcome the cross-domain problem with fewer auxiliary data requirements than baselines in the closed-world scenario. Moreover, the experimental results also demonstrate that differences in traffic features across datasets significantly impact cross-domain performance. The Wang dataset, collected in 2013, differs considerably from AWF in terms of collection time and TBB version. However, since TBB was still in its early stages, its traffic patterns were relatively simple, making it easier to distinguish between websites. As a result, in the 5-shot and 10-shot settings, few-shot models generally achieve better recognition performance on the Wang dataset than on the others. In contrast, the DF dataset, although collected in the same period as AWF, contains website traffic with lower discriminability, leading to inferior performance compared with the WF and Wang datasets. For the WF dataset, model performance shows clear polarization: methods such as DCWF, TF, and WFBDC perform poorly, while UDA-WF and NetCLR maintain high accuracy. The reason is primarily that the WF dataset differs substantially from AWF in both TBB versions and website content distribution, exacerbating cross-domain discrepancies and resulting in varied adaptability across models. These findings further highlight that discrepancies in traffic characteristics across datasets are the fundamental factor influencing cross-domain performance, and addressing them is essential for building robust WF attack models.

Remark 4: The experimental results demonstrate that UDA-WF can achieve state-of-the-art results across diverse scenarios and datasets, such as with different TBB versions.

D. Cross-Domain in Open-World Scenario

We also evaluate the performance of the UDA-WF model when facing the cross-domain problem in the open-world scenario. Due to more complex monitored requirements, we use AUC, precision, recall, and F1-score as evaluation metrics.

1) WF Dataset Evaluation Results: Table VII shows that UDA-WF outperforms the SOTA few-shot WF attacks on the WF dataset. The UDA-WF model achieves a precision of 95.24% and a recall of 91.67% under the 20-shot setting, where the prior SOTA attacks, WFBDC, DCWF, and NetCLR, achieve precisions of 67.52%, 75.65%, and 92.79%, and recalls of 74.77%, 66.52%, and 90.60%. Specifically, the average F1-score improvements of UDA-WF over DF, TF, WFBDC, DCWF, and NetCLR are about 103.61%, 199.67%, 52.03%, 55.37%, and 2.49%, respectively. Figure 9(a) shows that UDA-WF achieves a higher AUC of 0.9972 under the 20-shot setting, where SOTA few-shot methods, WFBDC, DCWF, and NetCLR, are about 0.8721, 0.8306, and 0.9526, respectively.

2) Wang Dataset Evaluation Results: Table VII shows that UDA-WF outperforms the SOTA few-shot WF attacks on the Wang dataset. The UDA-WF model achieves a precision of 94.83% and a recall of 94.46% under the 20-shot setting, while the prior SOTA attacks, WFBDC, DCWF, and NetCLR, achieve precisions of 92.88%, 93.52%, and 93.67%, and recalls of 92.04%, 92.11%, and 93.35%. Specifically, the average F1-score improvements of UDA-WF over DF, TF, WFBDC, DCWF, and NetCLR are about 41.12%, 37.67%, 3.26%, 2.52%, and 1.86%, respectively. Figure 9(b) shows that UDA-WF achieves the SOTA AUC of 0.9938 under the 20-shot setting, where SOTA few-shot methods, WFBDC, DCWF, and NetCLR, are about 0.9599, 0.9602, and 0.9665, respectively.

3) DF Dataset Evaluation Results: We further evaluate the performance on the DF dataset, and Table VII shows the results. The average F1-score improvements of UDA-WF over DF, TF, WFBDC, DCWF, and NetCLR are about 68.81%,

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:23:13 UTC from IEEE Xplore. Restrictions apply.

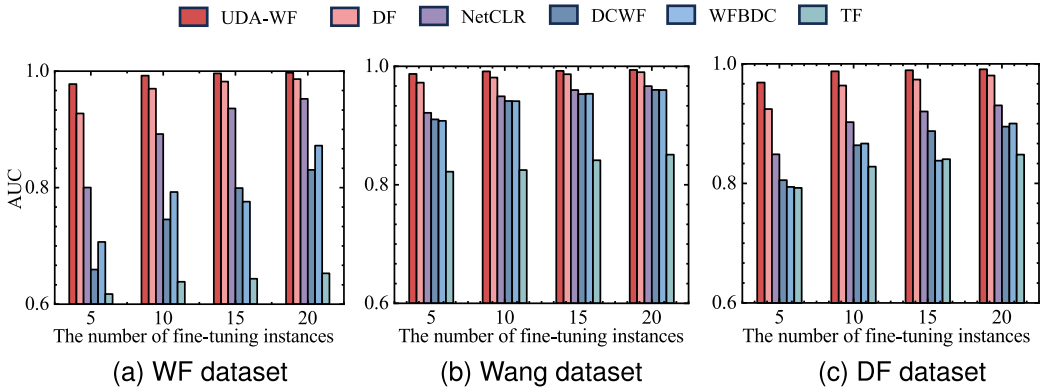


Fig. 9. Performance of few-shot WF attacks in the open-world scenario with different target domains.

TABLE VII
PERFORMANCE COMPARISON WITH SOTA FEW-SHOT WF ATTACKS IN THE OPEN-WORLD SCENARIO WITH DIFFERENT TARGET DOMAINS. ALL NUMBERS ARE %

Dataset	Methods	5-shot			10-shot			15-shot			20-shot		
		Pre	Recall	F1-score	Pre	Recall	F1-score	Pre	Recall	F1-score	Pre	Recall	F1-score
WF dataset	DF	24.55	18.10	19.54	49.20	42.61	44.78	66.47	64.68	64.91	71.95	67.63	69.28
	TF	25.42	24.25	24.82	28.17	27.43	27.69	29.96	29.42	28.85	31.33	31.40	30.77
	WFBDC	53.10	41.92	41.92	59.76	58.98	54.47	71.99	55.68	59.62	67.52	74.77	67.86
	DCWF	44.42	32.60	35.86	61.98	49.54	53.68	72.94	60.35	65.00	75.65	66.52	69.92
	NetCLR	77.25	60.36	65.88	88.93	78.62	82.59	91.80	87.36	89.14	92.79	90.60	91.49
	UDA-WF	**79.14**	**63.20**	**68.89**	**89.49**	**79.82**	**83.96**	**93.30**	**88.70**	**90.74**	**95.24**	**91.67**	**93.29**
Wang dataset	DF	51.00	49.72	49.34	64.74	64.82	64.26	74.10	73.12	73.06	80.73	79.45	79.66
	TF	59.42	64.81	61.10	71.64	65.17	65.76	73.35	68.36	68.82	73.91	70.40	70.51
	WFBDC	85.76	81.73	82.69	89.01	88.34	88.49	91.95	90.82	91.19	92.88	92.04	92.25
	DCWF	87.56	82.19	83.96	90.39	88.35	89.17	92.34	90.70	91.31	93.52	92.11	92.68
	NetCLR	85.90	84.36	84.41	90.15	89.86	89.75	92.22	92.05	91.96	93.67	93.35	93.30
	UDA-WF	**87.91**	**86.18**	**86.66**	**92.10**	**91.26**	**91.53**	**93.64**	**93.18**	**93.30**	**94.83**	**94.46**	**94.57**
DF dataset	DF	16.13	27.06	24.30	67.02	65.80	66.40	73.34	68.26	68.78	71.80	70.43	70.86
	TF	70.47	58.83	60.47	70.48	71.46	70.42	77.96	78.30	77.74	72.26	69.94	69.68
	WFBDC	71.83	59.24	62.64	78.55	73.65	75.29	81.87	77.11	78.76	83.48	80.28	81.43
	DCWF	75.73	65.50	65.13	85.20	73.04	77.35	88.69	77.68	82.02	90.17	79.27	83.65
	NetCLR	76.10	69.94	71.51	85.76	80.77	82.47	89.68	84.20	86.36	90.05	86.23	87.61
	UDA-WF	**77.23**	**69.97**	**72.52**	**87.16**	**80.83**	**83.32**	**90.44**	**84.73**	**87.16**	**90.98**	**86.32**	**88.30**

19.16%, 11.38%, 7.72%, and 1.03%. Figure 9(c) shows that UDA-WF achieves the SOTA AUC of 0.9909 under the 20-shot setting, improving over WFBDC, DCWF, and NetCLR by 10.09%, 10.71%, and 6.51%.

Remark 5: The results in the open-world scenario show that UDA-WF remains effective when facing the open-world scenario and achieves SOTA performance.

E. Few-Shot WF Attacks in Large Open-World

Since there are thousands of non-monitored websites in real scenarios, we evaluate the performance of the few-shot WF attacks when faced with a large number of non-monitored websites. We use 50k, 100k, 200k, and 400k non-monitored websites, all from the AWF dataset. We use AUC as our evaluation metric.

Figure 10 highlights that UDA-WF consistently outperforms other few-shot WF attacks across different fine-tuning instance settings in the large open-world scenario. For example, under the 5-shot setting, our UDA-WF model achieves an average AUC of approximately 0.9791, improving over the SOTA baselines, WFBDC, DCWF, and NetCLR, by about 18.00%, 21.05%, and 12.44%, respectively. Notably, when the number of fine-tuning instances remains constant, the AUC scores for all models show minimal variation across different large open-world non-monitored website settings. However, the AUC scores increase significantly as fine-tuning instances increase. For example, the average AUC of UDA-WF increases from 0.9791 under the 5-shot setting to 0.9934 under the 20-shot setting, achieving an improvement of 1.54%. The results show that the performance of few-shot WF attacks remains stable in diverse large open-world settings, while increasing fine-tuning instances can significantly enhance recognition performance.

Remark 6: In the large open-world scenario, our UDA-WF model achieves higher AUCs, indicating that a large number of non-monitored websites have less impact on our UDA-WF compared to other baselines.

F. Concept Drift

As time passes, the distribution difference between fine-tuning and testing website data increases, leading to a decline

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:23:13 UTC from IEEE Xplore. Restrictions apply.

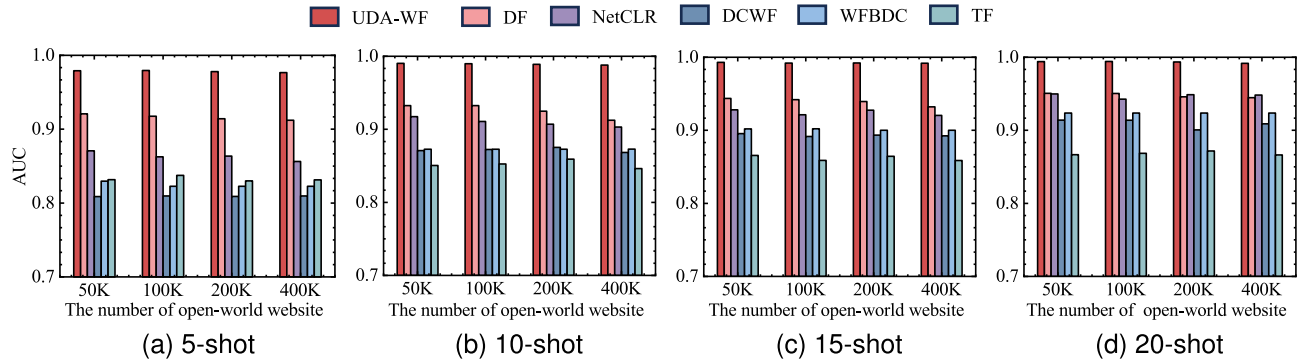


Fig. 10. Performance of few-shot WF attacks in the large open-world scenario.

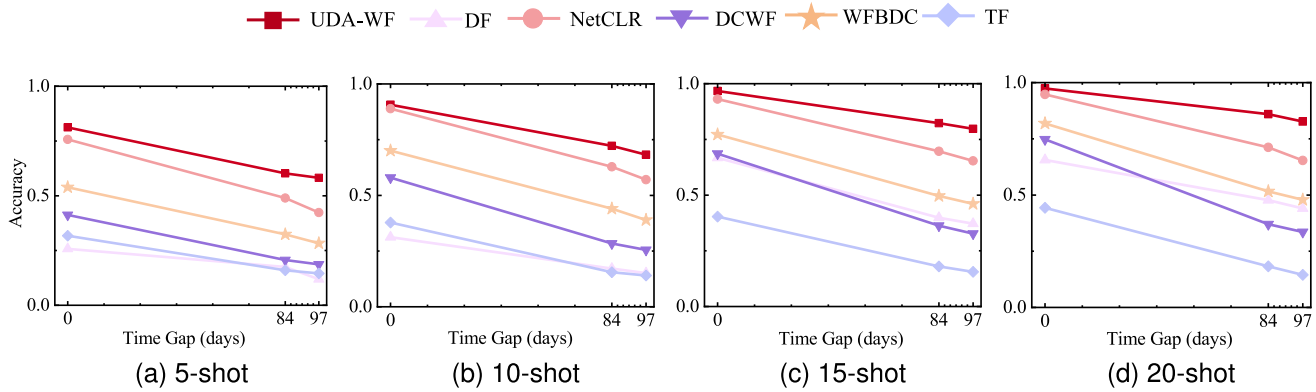


Fig. 11. Performance of few-shot WF attacks under the concept drift scenario caused by different time gaps.

in recognition performance. In this experiment, we evaluate the resilience of our UDA-WF and the baselines against concept drift on the WFdrift84 and WFdrift97 datasets. These two datasets are captured on the 84th and 97th days after the initial capture and contain the same website as the initial capture. We use the accuracy as our evaluation metric.

As shown in Figure 11, our UDA-WF achieves higher accuracies than other baselines. UDA-WF achieves accuracies of 60.23% on WFdrift84 and 58.17% on WFdrift97 under the 5-shot setting, where NetCLR achieves accuracies of 48.97% on WFdrift84 and 42.35% on WFdrift97. Even 97 days after the first collection, the average accuracy improvements of UDA-WF over DF, TF, WFBDC, DCWF, and NetCLR are approximately 236.19%, 392.17%, 105.40%, 167.30%, and 26.72%. Moreover, we notice that the more fine-tuning instances are used, the more resilient the model becomes to concept drift. For example, UDA-WF experiences a performance drop of 39.60% on WFdrift97 under the 5-shot setting, whereas the decline is reduced to only 17.75% under the 20-shot setting. The results show that increasing fine-tuning instances can enhance the ability of the UDA-WF against concept drift.

Remark 7: The experimental results demonstrate that UDA-WF has strong resilience against concept drift. Although the performance of UDA-WF may degrade over time, it can be fine-tuned with a small amount of data to maintain its reliability. The resilience may be attributed to the UDA with the softmatch mechanism, where the pure invariant feature space with less noise makes the model extract more resilient features to concept drift.

G. WF Defense

WF defense techniques alter the traffic patterns of website traces to counter WF attacks. Since the principles of defense methods are various, we evaluate the performance of our model when facing WF defense methods of different principles. Since the Front [27] and TrafficSliver [28] are representative defense methods with different principles, we use them to generate defense datasets from the WF dataset. The Front inserts highly random dummy packets at the first few seconds of each trace, and the TrafficSliver splits user traffic across multiple Tor entry nodes. Accuracy is the primary evaluation metric.

As shown in Table VIII, UDA-WF achieves a higher accuracy of 71.83% with the Trafficesliver defense method under the 20-shot setting, which improves over baseline models, DF, TF, WFBDC, DCWF, and NetCLR, by 89.82%, 112.64%, 43.91%, 16.36%, and 28.68%. Moreover, even using 5 instances per website to fine-tune, the UDA-WF achieves an accuracy of 37.05% when facing the Front. In contrast, the prior SOTA few-shot method, WFBDC, DCWF, and NetCLR, only achieves an accuracy of 6.17%, 7.82%, and 12.31%.

Remark 8: The experimental results demonstrate that our UDA-WF is more robust against WF defenses of different principles.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:23:13 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
PERFORMANCE COMPARISON OF FEW-SHOT WF ATTACKS AGAINST DIFFERENT WF DEFENSES. ACCURACY IS THE EVALUATION METRIC, AND ALL NUMBERS ARE %

K-shot	Methods	Defense		
		No defense	Front	TrafficSliver
5-shot	DF	25.74 ± 1.95	7.68 ± 0.88	21.75 ± 0.91
	TF	31.75 ± 0.27	5.57 ± 0.41	28.05 ± 0.82
	WFBDC	53.90 ± 0.20	6.17 ± 0.30	35.94 ± 1.22
	DCWF	41.25 ± 0.33	7.82 ± 0.26	35.30 ± 0.51
	NetCLR	75.71 ± 0.30	12.31 ± 0.17	51.52 ± 0.91
	UDA-WF	**81.21 ± 0.49**	**37.05 ± 0.20**	**54.06 ± 1.11**
10-shot	DF	31.23 ± 0.75	7.53 ± 0.80	20.92 ± 1.77
	TF	37.82 ± 0.28	5.46 ± 0.30	31.28 ± 0.96
	WFBDC	70.09 ± 0.15	6.70 ± 0.40	44.95 ± 0.48
	DCWF	57.93 ± 0.71	11.30 ± 0.23	49.92 ± 0.17
	NetCLR	88.89 ± 0.23	13.26 ± 0.61	49.77 ± 1.57
	UDA-WF	**90.68 ± 0.29**	**39.58 ± 0.23**	**53.33 ± 0.52**
15-shot	DF	66.70 ± 0.43	10.96 ± 0.54	42.28 ± 1.55
	TF	40.35 ± 0.60	5.65 ± 0.36	32.96 ± 0.72
	WFBDC	77.22 ± 0.14	6.77 ± 0.42	48.22 ± 1.08
	DCWF	68.60 ± 0.38	12.22 ± 0.34	57.74 ± 0.34
	NetCLR	93.12 ± 0.29	13.53 ± 0.25	50.54 ± 0.45
	UDA-WF	**96.70 ± 0.14**	**59.17 ± 0.36**	**59.72 ± 0.26**
20-shot	DF	65.57 ± 0.73	9.78 ± 0.46	37.84 ± 0.48
	TF	44.30 ± 0.14	5.91 ± 0.21	33.78 ± 0.76
	WFBDC	81.84 ± 0.08	6.51 ± 0.25	49.91 ± 0.14
	DCWF	74.76 ± 1.12	13.14 ± 0.58	61.73 ± 0.76
	NetCLR	94.72 ± 0.11	15.63 ± 0.29	55.82 ± 0.47
	UDA-WF	**97.37 ± 0.07**	**63.93 ± 0.43**	**71.83 ± 0.13**

VII. DISCUSSION AND FUTURE WORK

Our experimental results demonstrate that UDA-WF addresses cross-domain few-shot website fingerprinting with limited auxiliary data. However, several issues remain that warrant further investigation.

A. Exploring Few-Shot Multi-Tab WF Attacks

Similar to the existing few-shot WF attacks, our approach operates under the single-tab assumption. However, in real-world scenarios, users often browse with multiple open tabs, leading to overlapping traffic and reduced model accuracy. Although multi-tab WF attacks [39], [40], [41] have been proposed recently, they require substantial training data. Given our model's strength in few-shot learning, future research will alter the architectures to adapt to the multi-tab scenario.

B. Enhancing Feature Extraction Ability

Our model performs well on various data sets, but exhibits fluctuations due to the weak ability of the feature extractor to capture unique features on some datasets. Attention mechanisms [42], [43], [44] can enhance feature extraction by focusing on more discriminative features, but require large datasets, increasing bootstrap pre-training time. We will explore light-overhead attention structures to improve performance in real-world scenarios.

C. Handling Extreme Few-Shot Attack Scenario

We observe that UDA-WF sometimes underperforms in the 1-shot scenario, such as on the Wang and DF datasets. In this extreme setting, the limited number of target samples makes pseudo-labeling unreliable and prevents the establishment of stable inter-class constraints, ultimately reducing classification reliability. Thus, UDA-WF is suitable for scenarios with more than 5 shots. For the 1-shot scenario, we explore several strategies: replacing batch normalization of the feature extractor with group normalization to stabilize optimization with limited samples, increasing UDA instances to enlarge the invariant feature space, expanding auxiliary source categories to improve decision boundaries, and combining these approaches. While these strategies considerably enhance performance, UDA-WF remains slightly behind the best results on the Wang dataset. The inter-class metric methods [45], [46], [47] offer a potential solution by explicitly enforcing class separability. However, it typically requires constructing large numbers of pairs or triplets, substantially increasing training cost and data demands—contrary to the efficiency-oriented design of UDA-WF. We will explore a more efficient metric-based model with reduced data requirements to enhance UDA-WF for the 1-shot scenario.

VIII. CONCLUSION

This work proposes a data-efficient few-shot WF attack method, UDA-WF, based on UDA with the softmatch mechanism. First, UDA-WF pre-trains the feature extractor to fit the source feature space and decision boundaries, where GAP reduces feature dimensionality and classifier parameters, lowering auxiliary data requirements. Second, UDA-WF leverages UDA with the softmatch mechanism to generate and weight pseudo-labels from clustering unlabeled target data, guiding the feature extractor toward the intersection of source and target feature spaces. Finally, UDA-WF fine-tunes the feature extractor using a single-layer perceptron to extract the discriminative unique feature space of the target website dataset. We collect a comprehensive dataset over months, the WF dataset, to evaluate the performance of our UDA-WF. The experimental study shows that our UDA-WF can overcome the cross-domain problem with limited auxiliary data requirements and achieve the SOTA performance on three different datasets. It significantly overcomes the cross-domain problem while reducing auxiliary data requirements by 95% and pre-training bootstrap time by 99% compared to the SOTA method. UDA-WF achieves an accuracy of 97.37% under the 20-shot setting in the closed-world scenario and outperforms SOTA methods. To further demonstrate the model's adaptability to diverse real-world requirements, we validate it on the DF and Wang datasets, achieving accuracies exceeding 92% and 94%, respectively. Moreover, the results show that our UDA-WF is more resilient to concept drift and robust to WF defense. Our next step is to refine UDA-WF to enhance its feature extraction ability, handle the extreme few-shot attack scenario, and adapt to the multi-tab scenario.

REFERENCES

[1] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The second-generation onion router," in *Proc. USENIX Secur. Symp.*, 2004, pp. 1–17.
[2] T. Wang, X. Cai, R. Nithyanand, R. Johnson, and I. Goldberg, "Effective attacks and provable defenses for website fingerprinting," in *Proc. 23rd USENIX Secur. Symp. (USENIX Secur.)*, 2014, pp. 143–157.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:23:13 UTC from IEEE Xplore. Restrictions apply.

[3] J. Hayes and G. Danezis, "k-fingerprinting: A robust scalable website fingerprinting technique," in *Proc. 25th USENIX Secur. Symp. (USENIX Secur.)*, Aug. 2016, pp. 1187–1203.

[4] A. Panchenko et al., "Website fingerprinting at internet scale," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2016, pp. 1–15.

[5] V. Rimmer, D. Preuveneers, M. Juarez, T. V. Goethem, and W. Joosen, "Automated website fingerprinting through deep learning," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.

[6] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep fingerprinting: Undermining website fingerprinting defenses with deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Oct. 2018, pp. 1928–1943.

[7] M. S. Rahman, P. Sirinam, N. Mathews, K. G. Gangadhara, and M. Wright, "Tik-Tok: The utility of packet timing in website fingerprinting attacks," in *Proc. Privacy Enhancing Technol.*, 2020, pp. 5–24.

[8] M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du, "Subverting website fingerprinting defenses with robust traffic representation," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 607–624.

[9] F. Zhu, Y. Wang, C. Chen, J. Zhou, L. Li, and G. Liu, "Cross-domain recommendation: Challenges, progress, and prospects," in *Proc. 13th Int. Joint Conf. Artif. Intell.*, Aug. 2021, pp. 1–10.

[10] Y. Song, T. Wang, P. Cai, S. K. Mondal, and J. P. Sahoo, "A comprehensive survey of few-shot learning: Evolution, applications, challenges, and opportunities," *ACM Comput. Surv.*, vol. 55, no. 13s, pp. 1–40, Dec. 2023.

[11] P. Sirinam, N. Mathews, M. S. Rahman, and M. Wright, "Triplet fingerprinting: More practical and portable website fingerprinting with N-shot learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2019, pp. 1131–1148.

[12] C. Wang, J. Dani, X. Li, X. Jia, and B. Wang, "Adaptive fingerprinting: Website fingerprinting over few encrypted traffic," in *Proc. 11th ACM Conf. Data Appl. Secur. Privacy*, Apr. 2021, pp. 149–160.

[13] M. Chen, Y. Wang, and X. Zhu, "Few-shot website fingerprinting attack with meta-bias learning," *Pattern Recognit.*, vol. 130, pp. 1–14, Oct. 2022.

[14] H. Zou, J. Su, Z. Wei, S. Chen, and B. Zhao, "An efficient cross-domain few-shot website fingerprinting attack with Brownian distance covariance," *Comput. Netw.*, vol. 219, pp. 1–14, Dec. 2022.

[15] A. Bahramali, A. Bozorgi, and A. Houmansadr, "Realistic website fingerprinting by augmenting network traces," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2023, pp. 1035–1049.

[16] Q. Zhou, L. Wang, H. Zhu, and T. Lü, "Few-shot website fingerprinting attack with cluster adaptation," *Comput. Netw.*, vol. 229, Jun. 2023, Art. no. 109780.

[17] H. Chen et al., "SoftMatch: Addressing the quantity-quality trade-off in semi-supervised learning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023, pp. 1–21.

[18] X. Cai, X. C. Zhang, B. Joshi, and R. Johnson, "Touching from a distance: Website fingerprinting attacks and defenses," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2012, pp. 605–616.

[19] Q. Zhou, L. Wang, H. Zhu, T. Lu, and V. S. Sheng, "WF-transformer: Learning temporal features for accurate anonymous traffic identification by using transformer networks," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 30–43, 2023.

[20] B. Gao, W. Liu, G. Liu, F. Nie, and J. Huang, "Multi-level resource-coherented graph learning for website fingerprinting attacks," *IEEE Trans. Inf. Forensics Security*, vol. 20, pp. 693–708, 2025.

[21] N. Mathews, J. K. Holland, N. Hopper, and M. Wright, "Laserbeak: Evolving website fingerprinting attacks with attention and multi-channel feature representation," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 9285–9300, 2024.

[22] Y. Xie et al., "Contrastive fingerprinting: A novel website fingerprinting attack over few-shot traces," in *Proc. ACM Web Conf.*, May 2024, pp. 1203–1214.

[23] C. Luo, W. Tang, Q. Wang, and D. Zheng, "Few-shot website fingerprinting with distribution calibration," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 1, pp. 632–648, Jan. 2025.

[24] Q. Lyu, H. Xie, W. Wang, Y. Cheng, Y. Chen, and Z. Wang, "TFAN: A task-adaptive feature alignment network for few-shot website fingerprinting attacks on tor," *Comput. Secur.*, vol. 144, pp. 1–12, Sep. 2024.

[25] J. Tan, H. Wang, S. Han, D. Man, and W. Yang, "An adaptability-enhanced few-shot website fingerprinting attack based on collusion," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 8220–8235, 2024.

[26] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A systematic approach to developing and evaluating website fingerprinting defenses," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2014, pp. 227–238.

[27] J. Gong and T. Wang, "Zero-delay lightweight defenses against website fingerprinting," in *Proc. 29th USENIX Secur. Symp. (USENIX Secur.)*, 2020, pp. 717–734.

[28] W. De la Cadena et al., "TrafficSliver: Fighting website fingerprinting attacks with traffic splitting," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Nov. 2020, pp. 1971–1985.

[29] J. Gong, W. Zhang, C. Zhang, and T. Wang, "Surakav: Generating realistic traces for a strong website fingerprinting defense," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2022, pp. 1558–1573.

[30] Z. Ling, G. Xiao, L. Luo, R. Wang, X. Xu, and G. Liu, "WFGuard: An effective fuzzing-testing-based traffic morphing defense against website fingerprinting," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, May 2024, pp. 441–450.

[31] M. Jiang, B. Cui, J. Fu, T. Wang, L. Yao, and B. K. Bhargava, "RUDOLF: An efficient and adaptive defense approach against website fingerprinting attacks based on soft actor-critic algorithm," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 7794–7809, 2024.

[32] H. Venkateswara, J. Eusebio, S. Chakraborty, and S. Panchanathan, "Deep hashing network for unsupervised domain adaptation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 5385–5394.

[33] K. Saito, K. Watanabe, Y. Ushiku, and T. Harada, "Maximum classifier discrepancy for unsupervised domain adaptation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3723–3732.

[34] M. Litrico, A. Del Bue, and P. Morerio, "Guiding pseudo-labels with uncertainty estimation for source-free unsupervised domain adaptation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 7640–7650.

[35] I. Nassar, M. Hayat, E. Abbasnejad, H. Rezatofighi, and G. Haffari, "Protocon: pseudo-label refinement via online clustering and prototypical consistency for efficient semi-supervised learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2023, pp. 11641–11650.

[36] S. Liu, W. Cao, R. Fu, K. Yang, and Z. Yu, "RPSC: Robust pseudo-labeling for semantic clustering," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, vol. 38, 2024, pp. 14008–14016.

[37] J. Shi et al., "Dual pseudo-labels interactive self-training for semi-supervised visible-infrared person re-identification," in *Proc. IEEE Int. Conf. Compute. Vis. (ICCV)*, Oct. 2023, pp. 11218–11228.

[38] J. Liang, D. Hu, and J. Feng, "Do we really need to access the source data? Source hypothesis transfer for unsupervised domain adaptation," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 6028–6039.

[39] Z. Guan, G. Xiong, G. Gou, Z. Li, M. Cui, and C. Liu, "BAPM: Block attention profiling model for multi-tab website fingerprinting attacks on Tor," in *Proc. Annu. Comput. Secur. Appl. Conf.*, Dec. 2021, pp. 248–259.

[40] X. Deng et al., "Robust multi-tab website fingerprinting attacks in the wild," in *Proc. IEEE Symp. Secur. Privacy*, May 2023, pp. 1005–1022.

[41] Z. Jin, T. Lu, S. Luo, and J. Shang, "Transformer-based model for multi-tab website fingerprinting attack," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2023, pp. 1050–1064.

[42] A. Vaswani et al., "Attention is all you need," in *Proc. NIPS*, 2017, pp. 6000–6010.

[43] J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2017, pp. 7132–7141.

[44] Z. Huang, X. Wang, L. Huang, C. Huang, Y. Wei, and W. Liu, "CCNet: Criss-cross attention for semantic segmentation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 603–612.

[45] W. Huang, H. Zhang, and K. Zheng, "DiffKendall: A novel approach for few-shot learning with differentiable Kendall's rank correlation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 1–13.

[46] Y. Zhou, Y. Guo, S. Hao, and R. Hong, "Hierarchical prototype refinement with progressive inter-categorical discrimination maximization for few-shot learning," *IEEE Trans. Image Process.*, vol. 31, pp. 3414–3429, 2022.

[47] J. Xie, F. Long, J. Lv, Q. Wang, and P. Li, "Joint distribution matters: Deep Brownian distance covariance for few-shot classification," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2022, pp. 7972–7981.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 06:23:13 UTC from IEEE Xplore. Restrictions apply.